

CustomResourceDefinition 的版本

CustomResourceDefinition 的版本

本页介绍如何添加版本信息到 CustomResourceDefinitions。目的是标明 CustomResourceDefinitions 的稳定级别或者服务于 API 升级。API 升级时需要在不同 API 表示形式之间进行转换。本页还描述如何将对象从一个版本升级到另一个版本。

你应该对定制资源有一些初步了解。

概览 {#overview}

CustomResourceDefinition API 提供了引入和升级 CustomResourceDefinition 新版本所用的工作流程。

创建 CustomResourceDefinition 时，会在 CustomResourceDefinition spec.versions 列表设置适当的稳定级别和版本号。例如，v1beta1 表示第一个版本尚未稳定。所有定制资源对象将首先用这个版本保存。

创建 CustomResourceDefinition 后，客户端可以开始使用 v1beta1 API。

稍后可能需要添加新版本，例如 v1。

添加新版本：

1. 选择一种转化策略。由于定制资源对象需要能够两种版本都可用，这意味着它们有时会以与存储版本不同的版本来提供服务。为了能够做到这一点，有时必须在它们存储的版本和提供的版本之间进行转换。如果转换涉及模式变更，并且需要自定义逻辑，则应该使用 Webhook 来完成。如果没有模式变更，则可使用默认的 None 转换策略，为不同版本提供服务时只有 apiVersion 字段会被改变。
2. 如果使用转换 Webhook，请创建并部署转换 Webhook。更多详细信息请参见 Webhook 转换。
3. 更新 CustomResourceDefinition，将新版本设置为 served: true，加入到 spec.versions 列表。另外，还要设置 spec.conversion 字段为所选的转换策略。如果使用转换 Webhook，请配置 spec.conversion.webhookClientConfig 来调用 Webhook。

添加新版本后，客户端可以逐步迁移到新版本。让某些客户使用旧版本的同时支持其他人使用新版本是相当安全的。

将存储的对象迁移到新版本：

1. 请参阅将现有对象升级到新的存储版本节。

对于客户来说，在将对象升级到新的存储版本之前、期间和之后使用旧版本和新版本都是安全的。

删除旧版本：

1. 确保所有客户端都已完全迁移到新版本。可以查看 kube-apiserver 的日志以识别仍通过旧版本进行访问的所有客户端。
2. 在 spec.versions 列表中将旧版本的 served 设置为 false。如果仍有客户端意外地使用旧版本，他们可能开始会报告采用旧版本尝试访问定制资源的错误消息。如果发生这种情况，请将旧版本的 served: true 恢复，然后迁移余下的客户端使用新版本，然后重复此步骤。
3. 确保已完成将现有对象升级到新存储版本的步骤。
 1. 在 CustomResourceDefinition 的 spec.versions 列表中，确认新版本的 storage 已被设置为 true。
 2. 确认旧版本不在 CustomResourceDefinition status.storedVersions 中。
4. 从 CustomResourceDefinition spec.versions 列表中删除旧版本。
5. 在转换 Webhooks 中放弃对旧版本的转换支持。

指定多个版本 {#specify-multiple-versions}

CustomResourceDefinition API 的 versions 字段可用于支持你所开发的定制资源的多个版本。版本可以具有不同的模式，并且转换 Webhook 可以在多个版本之间转换定制资源。在适当的情况下，Webhook 转换应遵循 Kubernetes API 约定。具体来说，请查阅 API 变更文档 以获取一些有用的问题和建议。

在 apiextensions.k8s.io/v1beta1 版本中曾经有一个 version 字段，名字不叫做 versions。该 version 字段已经被废弃，成为可选项。不过如果该字段不是空，则必须与 versions 字段中的第一个条目匹配。

下面的示例显示了两个版本的 CustomResourceDefinition。第一个例子中假设所有的版本使用相同的模式而它们之间没有转换。YAML 中的注释提供了更多背景信息。

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
```

metadata:

name 必须匹配后面 spec 中的字段，且使用格式。

name: crontabs.example.com

spec:

组名，用于 REST API: /apis//

group: example.com

此 CustomResourceDefinition 所支持的版本列表

versions:

- name: v1beta1

每个 version 可以通过 served 标志启用或禁止

served: true

有且只能有一个 version 必须被标记为存储版本

storage: true

schema 是必需字段

schema:

openAPIV3Schema:

type: object

properties:

host:

type: string

port:

type: string

- name: v1

served: true

storage: false

schema:

openAPIV3Schema:

type: object

properties:

host:

type: string

port:

type: string

conversion 节是 Kubernetes 1.13+ 版本引入的，其默认值为无转换，即 strategy 子字段设置为 None。

conversion:

None 转换假定所有版本采用相同的模式定义，仅仅将定制资源的 apiVersion 设置为合适的值。

strategy: None

可以是 Namespaced 或 Cluster

```
scope: Namespaced
names:
  # 名称的复数形式, 用于 URL: /apis///
  plural: crontabs
  # 名称的单数形式, 用于在命令行接口和显示时作为其别名
  singular: crontab
  # kind 通常是驼峰编码 (CamelCased) 的单数形式, 用于资源清单中
  kind: CronTab
  # shortNames 允许你在命令行接口中使用更短的字符串来匹配你的资源
  shortNames:
    - ct
```

在 v1.16 中被弃用以推荐使用 apiextensions.k8s.io/v1

```
apiVersion: apiextensions.k8s.io/v1beta1
```

```
kind: CustomResourceDefinition
```

```
metadata:
```

name 必须匹配后面 spec 中的字段, 且使用格式.

```
name: crontabs.example.com
```

```
spec:
```

组名, 用于 REST API: /apis//

```
group: example.com
```

此 CustomResourceDefinition 所支持的版本列表

```
versions:
```

```
- name: v1beta1
```

每个 version 可以通过 served 标志启用或禁止

```
served: true
```

有且只能有一个 version 必须被标记为存储版本

```
storage: true
```

```
- name: v1
```

```
served: true
```

```
storage: false
```

```
validation:
```

```
openAPIV3Schema:
```

```
  type: object
```

```
  properties:
```

```
    host:
```

```
      type: string
```

```
    port:
```

```
      type: string
```

conversion 节是 Kubernetes 1.13+ 版本引入的, 其默认值为无转换, 即 strategy 子字段设置为 None。

conversion:

None 转换假定所有版本采用相同的模式定义，仅仅将定制资源的 `apiVersion` 设置为合适的值。

strategy: `None`

可以是 `Namespaced` 或 `Cluster`

scope: `Namespaced`

names:

名称的复数形式，用于 URL: `/apis///`

plural: `crontabs`

名称的单数形式，用于在命令行接口和显示时作为其别名

singular: `crontab`

kind 通常是大驼峰编码 (`PascalCased`) 的单数形式，用于资源清单中

kind: `CronTab`

`shortNames` 允许你在命令行接口中使用更短的字符串来匹配你的资源

shortNames:

`- ct`

你可以将 CustomResourceDefinition 存储在 YAML 文件中，然后使用 `kubectl apply` 来创建它。

```
kubectl apply -f my-versioned-crontab.yaml
```

在创建之后，API 服务器开始在 HTTP REST 端点上为每个已启用的版本提供服务。在上面的示例中，API 版本可以在 `/apis/example.com/v1beta1` 和 `/apis/example.com/v1` 处获得。

版本优先级 {#version-priority}

不考虑 CustomResourceDefinition 中版本被定义的顺序，`kubectl` 使用具有最高优先级的版本作为访问对象的默认版本。优先级是通过解析 **name** 字段来确定版本号、稳定性（GA、Beta 或 Alpha）以及该稳定性级别内的序列。

用于对版本进行排序的算法在设计上与 Kubernetes 项目对 Kubernetes 版本进行排序的方式相同。版本以 `v` 开头跟一个数字，一个可选的 `beta` 或者 `alpha` 和一个可选的附加数字作为版本信息。从广义上讲，版本字符串可能看起来像 `v2` 或者 `v2beta1`。使用以下算法对版本进行排序：

- 遵循 Kubernetes 版本模式的条目在不符合条件的条目之前进行排序。
- 对于遵循 Kubernetes 版本模式的条目，版本字符串的数字部分从最大到最小排序。

- 如果第一个数字后面有字符串 beta 或 alpha，它们首先按去掉 beta 或 alpha 之后的版本号排序（相当于 GA 版本），之后按 beta 先、alpha 后的顺序排序，
- 如果 beta 或 alpha 之后还有另一个数字，那么也会针对这些数字从大到小排序。
- 不符合上述格式的字符串按字母顺序排序，数字部分不经过特殊处理。请注意，在下面的示例中，foo1 排在 foo10 之前。这与遵循 Kubernetes 版本模式的条目的数字部分排序不同。

如果查看以下版本排序列表，这些规则就容易懂了：

```
- v10
- v2
- v1
- v11beta2
- v10beta3
- v3beta1
- v12alpha1
- v11alpha2
- foo1
- foo10
```

对于指定多个版本中的示例，版本排序顺序为 v1，后跟着 v1beta1。这导致了 kubectl 命令使用 v1 作为默认版本，除非所提供的对象指定了版本。

版本废弃 {#version-deprecation}

从 v1.19 开始，CustomResourceDefinition 可以指示其定义的资源特定版本已废弃。当该资源的已废弃版本发出 API 请求时，会在 API 响应中以报头的形式返回警告消息。如果需要，可以自定义每个不推荐使用的资源版本的警告消息。

定制警告消息应该标明废弃的 API 组、版本和类别（kind），并且应该标明应该使用（如果有的话）哪个 API 组、版本和类别作为替代。

apiVersion: apiextensions.k8s.io/v1

kind: CustomResourceDefinition

metadata:

name: crontabs.example.com

spec:

group: example.com

names:

plural: crontabs

singular: crontab

```
kind: CronTab
scope: Namespaced
versions:
- name: v1alpha1
  served: true
  storage: false
  # 此属性标明此定制资源的 v1alpha1 版本已被弃用。
  # 发给此版本的 API 请求会在服务器响应中收到警告消息头。
  deprecated: true
  # 此属性设置用来覆盖返回给发送 v1alpha1 API 请求的客户端的默认警告信息。
  deprecationWarning: "example.com/v1alpha1 CronTab is deprecated; see
http://example.com/v1alpha1-v1 for instructions to migrate to example.com/v1 CronTab"
  schema: ...
- name: v1beta1
  served: true
  # 此属性标明该定制资源的 v1beta1 版本已被弃用。
  # 发给此版本的 API 请求会在服务器响应中收到警告消息头。
  # 针对此版本的请求所返回的是默认的警告消息。
  deprecated: true
  schema: ...
- name: v1
  served: true
  storage: true
  schema: ...

# 在 v1.16 中弃用以推荐使用 apiextensions.k8s.io/v1
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
metadata:
  name: crontabs.example.com
spec:
  group: example.com
  names:
    plural: crontabs
    singular: crontab
    kind: CronTab
  scope: Namespaced
  validation: ...
  versions:
  - name: v1alpha1
    served: true
```

```

storage: false
# 此属性标明此定制资源的 v1alpha1 版本已被弃用。
# 发给此版本的 API 请求会在服务器响应中收到警告消息头。
deprecated: true
# 此属性设置用来覆盖返回给发送 v1alpha1 API 请求的客户端的默认警告信息。
deprecationWarning: "example.com/v1alpha1 CronTab is deprecated; see
http://example.com/v1alpha1-v1 for instructions to migrate to example.com/v1 CronTab"
- name: v1beta1
  served: true
  # 此属性标明该定制资源的 v1beta1 版本已被弃用。
  # 发给此版本的 API 请求会在服务器响应中收到警告消息头。
  # 针对此版本的请求所返回的是默认警告消息。
  deprecated: true
- name: v1
  served: true
  storage: true

```

版本删除 {#version-removal}

在为所有提供旧版本自定义资源的集群将现有存储数据迁移到新 API 版本，并且从 CustomResourceDefinition 的 status.storedVersions 中删除旧版本之前，无法从 CustomResourceDefinition 清单文件中删除旧 API 版本。

```

apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: crontabs.example.com
spec:
  group: example.com
  names:
    plural: crontabs
    singular: crontab
    kind: CronTab
  scope: Namespaced
  versions:
    - name: v1beta1
      # 此属性标明该自定义资源的 v1beta1 版本已不再提供。
      # 发给此版本的 API 请求会在服务器响应中收到未找到的错误。
      served: false
      schema: ...
    - name: v1

```



```
served: true
# 新提供的版本应该设置为存储版本。
storage: true
schema: ...
```

Webhook 转换 {#webhook-conversion}

Webhook 转换在 Kubernetes 1.13 版本作为 Alpha 功能引入，在 Kubernetes 1.15 版本中成为 Beta 功能。要使用此功能，应启用 CustomResourceWebhookConversion 特性。在大多数集群上，这类 Beta 特性应该是自动启用的。请参阅特性门控文档以获得更多信息。

上面的例子在版本之间有一个 None 转换，它只在转换时设置 apiVersion 字段而不改变对象的其余部分。API 服务器还支持在需要转换时调用外部服务的 webhook 转换。例如：

- 定制资源的请求版本与其存储版本不同。
- 使用某版本创建了 Watch 请求，但所更改对象以另一版本存储。
- 定制资源的 PUT 请求所针对版本与存储版本不同。

为了涵盖所有这些情况并优化 API 服务器所作的转换，转换请求可以包含多个对象，以便减少外部调用。Webhook 应该独立执行各个转换。

编写一个转换 Webhook 服务器 {#write-a-conversion-webhook-server}

请参考定制资源转换 Webhook 服务器的实现；该实现在 Kubernetes e2e 测试中得到验证。Webhook 处理由 API 服务器发送的 ConversionReview 请求，并在 ConversionResponse 中封装发回转换结果。请注意，请求包含需要独立转换的定制资源列表，这些对象在被转换之后不能改变其在列表中的顺序。该示例服务器的组织方式使其可以复用于其他转换。大多数常见代码都位于 framework 文件中，只留下一个函数用于实现不同的转换。

转换 Webhook 服务器示例中将 ClientAuth 字段设置为空，默认为 NoClientCert。这意味着 webhook 服务器没有验证客户端（也就是 API 服务器）的身份。如果你需要双向 TLS 或者其他方式来验证客户端，请参阅如何验证 API 服务。

被允许的变更

转换 Webhook 不可以更改被转换对象的 metadata 中除 labels 和 annotations 之外的任何属性。尝试更改 name、UID 和 namespace 时都会导致引起转换的请求失败。所有其他变更都被忽略。

部署转换 Webhook 服务 {#deploy-the-conversion-webhook-service}

用于部署转换 Webhook 的文档与准入 Webhook 服务示例相同。这里的假设是转换 Webhook 服务器被部署为 default 名字空间中名为 example-conversion-webhook-server 的服务，并在路径 /crdconvert 上处理请求。

当 Webhook 服务器作为一个服务被部署到 Kubernetes 集群中时，它必须通过端口 443 公开其服务（服务器本身可以使用任意端口，但是服务对象应该将它映射到端口 443）。如果为服务器使用不同的端口，则 API 服务器和 Webhook 服务器之间的通信可能会失败。

配置 CustomResourceDefinition 以使用转换 Webhook {#configure-crd-to-use-conversion-webhooks}

通过修改 spec 中的 conversion 部分，可以扩展 None 转换示例来使用转换 Webhook。

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  # name 必须匹配后面 spec 中的字段，且使用格式.
  name: crontabs.example.com
spec:
  # 组名，用于 REST API: /apis//
  group: example.com
  # 此 CustomResourceDefinition 所支持的版本列表
  versions:
    - name: v1beta1
      # 每个 version 可以通过 served 标志启用或禁止
      served: true
      # 有且只能有一个 version 必须被标记为存储版本
      storage: true
      # 当不存在顶级模式定义时，每个版本（version）可以定义其自身的模式
      schema:
        openAPIV3Schema:
          type: object
          properties:
            hostPort:
              type: string
    - name: v1
      served: true
      storage: false
```

```

schema:
  openAPIV3Schema:
    type: object
    properties:
      host:
        type: string
      port:
        type: string
conversion:
  # Webhook strategy 告诉 API 服务器调用外部 Webhook 来完成定制资源之间的转换
  strategy: Webhook
  # 当 strategy 为 "Webhook" 时, webhook 属性是必需的, 该属性配置将被 API 服务器调用的
  # Webhook 端点
  webhook:
    # conversionReviewVersions 标明 Webhook 所能理解或偏好使用的
    # ConversionReview 对象版本。
    # API 服务器所能理解的列表中的第一个版本会被发送到 Webhook
    # Webhook 必须按所接收到的版本响应一个 ConversionReview 对象
    conversionReviewVersions: ["v1", "v1beta1"]
  clientConfig:
    service:
      namespace: default
      name: example-conversion-webhook-server
      path: /crdconvert
      caBundle: "Ci0tLS0tQk.....tLS0K"
  # 可以是 Namespaced 或 Cluster
  scope: Namespaced
  names:
    # 名称的复数形式, 用于 URL: /apis///
    plural: crontabs
    # 名称的单数形式, 用于在命令行接口和显示时作为其别名
    singular: crontab
    # kind 通常是驼峰编码 (CamelCased) 的单数形式, 用于资源清单中
    kind: CronTab
    # shortNames 允许你在命令行接口中使用更短的字符串来匹配你的资源
    shortNames:
      - ct

  # 在 v1.16 中被弃用以推荐使用 apiextensions.k8s.io/v1
  apiVersion: apiextensions.k8s.io/v1beta1
  kind: CustomResourceDefinition

```

```
metadata:
  # name 必须匹配后面 spec 中的字段，且使用格式.
  name: crontabs.example.com
spec:
  # 组名，用于 REST API: /apis//
  group: example.com
  # 裁剪掉下面的 OpenAPI 模式中未曾定义的对象字段
  preserveUnknownFields: false
  # 此 CustomResourceDefinition 所支持的版本列表
  versions:
    - name: v1beta1
      # 每个 version 可以通过 served 标志启用或禁止
      served: true
      # 有且只能有一个 version 必须被标记为存储版本
      storage: true
      # 当不存在顶级模式定义时，每个版本（version）可以定义其自身的模式
      schema:
        openAPIV3Schema:
          type: object
          properties:
            hostPort:
              type: string
    - name: v1
      served: true
      storage: false
      schema:
        openAPIV3Schema:
          type: object
          properties:
            host:
              type: string
            port:
              type: string
  conversion:
    # Webhook strategy 告诉 API 服务器调用外部 Webhook 来完成定制资源
    strategy: Webhook
    # 当 strategy 为 "Webhook" 时，webhookClientConfig 属性是必需的
    # 该属性配置将被 API 服务器调用的 Webhook 端点
    webhookClientConfig:
      service:
```

```

namespace: default
name: example-conversion-webhook-server
path: /crdconvert
caBundle: "Ci0tLS0tQk.....tLS0K"
# 可以是 Namespaced 或 Cluster
scope: Namespaced
names:
# 名称的复数形式, 用于 URL: /apis///
plural: crontabs
# 名称的单数形式, 用于在命令行接口和显示时作为其别名
singular: crontab
# kind 通常是驼峰编码 (CamelCased) 的单数形式, 用于资源清单中
kind: CronTab
# shortNames 允许你在命令行接口中使用更短的字符串来匹配你的资源
shortNames:
- ct

```

你可以将 CustomResourceDefinition 保存在 YAML 文件中，然后使用 `kubectl apply` 来应用它。

```
kubectl apply -f my-versioned-crontab-with-conversion.yaml
```

在应用新更改之前，请确保转换服务器已启动并正在运行。

调用 Webhook {#contacting-the-webhook}

API 服务器一旦确定请求应发送到转换 Webhook，它需要知道如何调用 Webhook。这是在 `webhookClientConfig` 中指定的 Webhook 配置。

转换 Webhook 可以通过 URL 或服务引用来调用，并且可以选择包含自定义 CA 包，以用于验证 TLS 连接。

URL

url 以标准 URL 形式给出 Webhook 的位置 (`scheme://host:port/path`)。host 不应引用集群中运行的服务，而应通过指定 `service` 字段来提供服务引用。在某些 API 服务器中，host 可以通过外部 DNS 进行解析（即 kube-apiserver 无法解析集群内 DNS，那样会违反分层规则）。host 也可以是 IP 地址。

请注意，除非你非常小心地在所有运行着可能调用 Webhook 的 API 服务器的主机上运行此 Webhook，否则将 `localhost` 或 `127.0.0.1` 用作 host 是风险很大的。这样的安装可能是不可移植的，或者不容易在一个新的集群中运行。

HTTP 协议必须为 https；URL 必须以 https:// 开头。

尝试使用用户或基本身份验证（例如，使用 user:password@）是不允许的。URL 片段（#...）和查询参数（?...）也是不允许的。

下面是为调用 URL 来执行转换 Webhook 的示例，其中期望使用系统信任根来验证 TLS 证书，因此未指定 caBundle：

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
...
spec:
  ...
  conversion:
    strategy: Webhook
    webhook:
      clientConfig:
        url: "https://my-webhook.example.com:9443/my-webhook-path"
  ...

# 在 v1.16 中已弃用以推荐使用 apiextensions.k8s.io/v1
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
...
spec:
  ...
  conversion:
    strategy: Webhook
    webhookClientConfig:
      url: "https://my-webhook.example.com:9443/my-webhook-path"
  ...
```

服务引用 {#service-reference}

webhookClientConfig 内部的 service 段是对转换 Webhook 服务的引用。如果 Webhook 在集群中运行，则应使用 service 而不是 url。服务的名字空间和名称是必需的。端口是可选的，默认为 443。路径是可选的，默认为/。

下面配置中，服务配置为在端口 1234、子路径 /my-path 上被调用。例子中针对 ServerName my-service-name.my-service-namespace.svc，使用自定义 CA 包验证 TLS 连接。

```

apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
...
spec:
...
conversion:
  strategy: Webhook
  webhook:
    clientConfig:
      service:
        namespace: my-service-namespace
        name: my-service-name
        path: /my-path
        port: 1234
        caBundle: "Ci0tLS0tQk.....tLS0K"
...

# v1.16 中被弃用以推荐使用 apiextensions.k8s.io/v1
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
...
spec:
...
conversion:
  strategy: Webhook
  webhookClientConfig:
    service:
      namespace: my-service-namespace
      name: my-service-name
      path: /my-path
      port: 1234
      caBundle: "Ci0tLS0tQk.....tLS0K"
...

```

Webhook 请求和响应 {#webhook-request-and-response}

请求 {#request}

向 Webhook 发起请求的动词是 POST，请求的 Content-Type 为 application/json。请求的主题为 JSON 序列化形式的 apiextensions.k8s.io API 组的 ConversionReview API 对象。

Webhook 可以在其 CustomResourceDefinition 中使用 conversionReviewVersions 字段设置它们接受的 ConversionReview 对象的版本：

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
...
spec:
  ...
  conversion:
    strategy: Webhook
    webhook:
      conversionReviewVersions: ["v1", "v1beta1"]
  ...
```

创建 apiextensions.k8s.io/v1 版本的自定义资源定义时，conversionReviewVersions 是必填字段。Webhook 要求支持至少一个 ConversionReview 当前和以前的 API 服务器可以理解的版本。

```
# v1.16 已弃用以推荐使用 apiextensions.k8s.io/v1
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
...
spec:
  ...
  conversion:
    strategy: Webhook
    conversionReviewVersions: ["v1", "v1beta1"]
  ...
```

创建 apiextensions.k8s.io/v1beta1 定制资源定义时若未指定 conversionReviewVersions，则默认值为 v1beta1。

API 服务器将 conversionReviewVersions 列表中他们所支持的第一个 ConversionReview 资源版本发送给 Webhook。如果列表中的版本都不被 API 服务器支持，则无法创建自定义资源定义。如果某 API 服务器遇到之前创建的转换 Webhook 配置，并且该配置不支持 API 服务器知道如何发送的任何 ConversionReview 版本，调用 Webhook 的尝试会失败。

下面的示例显示了包含在 ConversionReview 对象中的数据，该请求意在将 CronTab 对象转换为 example.com/v1：

```
{
  "apiVersion": "apiextensions.k8s.io/v1",
```



```

"kind": "ConversionReview",
"request": {
  # 用来唯一标识此转换调用的随机 UID
  "uid": "705ab4f5-6393-11e8-b7cc-42010a800002",

  # 对象要转换到的目标 API 组和版本
  "desiredAPIVersion": "example.com/v1",

  # 要转换的对象列表，其中可能包含一个或多个对象，版本可能相同也可能不同
  "objects": [
    {
      "kind": "CronTab",
      "apiVersion": "example.com/v1beta1",
      "metadata": {
        "creationTimestamp": "2019-09-04T14:03:02Z",
        "name": "local-crontab",
        "namespace": "default",
        "resourceVersion": "143",
        "uid": "3415a7fc-162b-4300-b5da-fd6083580d66"
      },
      "hostPort": "localhost:1234"
    },
    {
      "kind": "CronTab",
      "apiVersion": "example.com/v1beta1",
      "metadata": {
        "creationTimestamp": "2019-09-03T13:02:01Z",
        "name": "remote-crontab",
        "resourceVersion": "12893",
        "uid": "359a83ec-b575-460d-b553-d859cedde8a0"
      },
      "hostPort": "example.com:2345"
    }
  ]
}

{
  # v1.16 中已废弃以推荐使用 apiextensions.k8s.io/v1
  "apiVersion": "apiextensions.k8s.io/v1beta1",

```

```
"kind": "ConversionReview",
"request": {
  # 用来唯一标识此转换调用的随机 UID
  "uid": "705ab4f5-6393-11e8-b7cc-42010a800002",

  # 对象要转换到的目标 API 组和版本
  "desiredAPIVersion": "example.com/v1",

  # 要转换的对象列表，其中可能包含一个或多个对象，版本可能相同也可能不同
  "objects": [
    {
      "kind": "CronTab",
      "apiVersion": "example.com/v1beta1",
      "metadata": {
        "creationTimestamp": "2019-09-04T14:03:02Z",
        "name": "local-crontab",
        "namespace": "default",
        "resourceVersion": "143",
        "uid": "3415a7fc-162b-4300-b5da-fd6083580d66"
      },
      "hostPort": "localhost:1234"
    },
    {
      "kind": "CronTab",
      "apiVersion": "example.com/v1beta1",
      "metadata": {
        "creationTimestamp": "2019-09-03T13:02:01Z",
        "name": "remote-crontab",
        "resourceVersion": "12893",
        "uid": "359a83ec-b575-460d-b553-d859cedde8a0"
      },
      "hostPort": "example.com:2345"
    }
  ]
}
```

响应 {#response}

Webhook 响应包含 200 HTTP 状态码、Content-Type: application/json，在主体中包含 JSON 序列化形式的数据，在 response 节中给出 ConversionReview 对象（与发送的版本相同）。

如果转换成功，则 Webhook 应该返回包含以下字段的 response 节：

- uid，从发送到 webhook 的 request.uid 复制而来
- result，设置为 {"status":"Success"}
- convertedObjects，包含来自 request.objects 的所有对象，均已转换为 request.desiredAPIVersion

Webhook 的最简单成功响应示例：

```
{
  "apiVersion": "apiextensions.k8s.io/v1",
  "kind": "ConversionReview",
  "response": {
    # 必须与 匹配
    "uid": "705ab4f5-6393-11e8-b7cc-42010a800002",
    "result": {
      "status": "Success"
    },
    # 这里的对象必须与 request.objects 中的对象顺序相同并且其 apiVersion 被设置为。
    # kind、metadata.uid、metadata.name 和 metadata.namespace 等字段都不可被
    Webhook 修改。
    # Webhook 可以更改 metadata.labels 和 metadata.annotations 字段值。
    # Webhook 对 metadata 中其他字段的更改都会被忽略
    "convertedObjects": [
      {
        "kind": "CronTab",
        "apiVersion": "example.com/v1",
        "metadata": {
          "creationTimestamp": "2019-09-04T14:03:02Z",
          "name": "local-crontab",
          "namespace": "default",
          "resourceVersion": "143",
          "uid": "3415a7fc-162b-4300-b5da-fd6083580d66"
        },
        "host": "localhost",
```

```

    "port": "1234"
  },
  {
    "kind": "CronTab",
    "apiVersion": "example.com/v1",
    "metadata": {
      "creationTimestamp": "2019-09-03T13:02:01Z",
      "name": "remote-crontab",
      "resourceVersion": "12893",
      "uid": "359a83ec-b575-460d-b553-d859cedde8a0"
    },
    "host": "example.com",
    "port": "2345"
  }
]
}
}

{
  # v1.16 中已弃用以推荐使用 apiextensions.k8s.io/v1
  "apiVersion": "apiextensions.k8s.io/v1beta1",
  "kind": "ConversionReview",
  "response": {
    # 必须与 匹配
    "uid": "705ab4f5-6393-11e8-b7cc-42010a800002",
    "result": {
      "status": "Failed"
    },
  },
  # 这里的对象必须与 request.objects 中的对象顺序相同并且其 apiVersion 被设置为。
  # kind、metadata.uid、metadata.name 和 metadata.namespace 等字段都不可被
  Webhook 修改。
  # Webhook 可以更改 metadata.labels 和 metadata.annotations 字段值。
  # Webhook 对 metadata 中其他字段的更改都会被忽略。
  "convertedObjects": [
    {
      "kind": "CronTab",
      "apiVersion": "example.com/v1",
      "metadata": {
        "creationTimestamp": "2019-09-04T14:03:02Z",
        "name": "local-crontab",
        "namespace": "default",

```

```

    "resourceVersion": "143",
    "uid": "3415a7fc-162b-4300-b5da-fd6083580d66"
  },
  "host": "localhost",
  "port": "1234"
},
{
  "kind": "CronTab",
  "apiVersion": "example.com/v1",
  "metadata": {
    "creationTimestamp": "2019-09-03T13:02:01Z",
    "name": "remote-crontab",
    "resourceVersion": "12893",
    "uid": "359a83ec-b575-460d-b553-d859cedde8a0"
  },
  "host": "example.com",
  "port": "2345"
}
]
}
}

```

如果转换失败，则 Webhook 应该返回包含以下字段的 response 节：

- uid，从发送到 Webhook 的 request.uid 复制而来
- result，设置为 {"status": "Failed"}

转换失败会破坏对定制资源的读写访问，包括更新或删除资源的能力。转换失败应尽可能避免，并且不可用于实施合法性检查约束（应改用验证模式或 Webhook 准入插件）。

来自 Webhook 的响应示例，指示转换请求失败，并带有可选消息：

```

{
  "apiVersion": "apiextensions.k8s.io/v1",
  "kind": "ConversionReview",
  "response": {
    "uid": "",
    "result": {
      "status": "Failed",
      "message": "hostPort could not be parsed into a separate host and port"
    }
  }
}

```

```

    }
  }
}

{
  # v1.16 中弃用以推荐使用 apiextensions.k8s.io/v1
  "apiVersion": "apiextensions.k8s.io/v1beta1",
  "kind": "ConversionReview",
  "response": {
    "uid": "",
    "result": {
      "status": "Failed",
      "message": "hostPort could not be parsed into a separate host and port"
    }
  }
}

```

编写、读取和更新版本化的 CustomResourceDefinition 对象 {#write-read-and-update-versioned-crd-objects}

写入对象时，将存储为写入时指定的存储版本。如果存储版本发生变化，现有对象永远不会被自动转换。然而，新创建或被更新的对象将以新的存储版本写入。对象写入的版本不再被支持是有可能的。

当读取对象时，你需要在路径中指定版本。你可以请求当前提供的任意版本的对象。如果所指定的版本与对象的存储版本不同，Kubernetes 会按所请求的版本将对象返回，但磁盘上存储的对象不会更改。

在为读取请求提供服务时正返回的对象会发生什么取决于 CRD 的 `spec.conversion` 中指定的内容：

- 如果所指定的 `strategy` 值是默认的 `None`，则针对对象的唯一修改是更改其 `apiVersion` 字符串，并且可能修剪未知字段（取决于配置）。请注意，如果存储和请求版本之间的模式不同，这不太可能导致好的结果。尤其是如果在相同的数据类不同版本中采用不同字段来表示时，不应使用此策略。
- 如果指定了 `Webhook` 转换，则此机制将控制转换。

如果你更新一个现有对象，它将以当前的存储版本被重写。这是可以将对象从一个版本改到另一个版本的唯一办法。

为了说明这一点，请考虑以下假设的一系列事件：

1. 存储版本是 v1beta1。你创建一个对象。该对象以版本 v1beta1 存储。
2. 你将为 CustomResourceDefinition 添加版本 v1，并将其指定为存储版本。此处 v1 和 v1beta1 的模式是相同的，这通常是在 Kubernetes 生态系统中将 API 提升为稳定版时的情况。
3. 你使用版本 v1beta1 来读取你的对象，然后你再次用版本 v1 读取对象。除了 apiVersion 字段之外，返回的两个对象是完全相同的。
4. 你创建一个新对象。该对象存储为版本 v1。你现在有两个对象，其中一个是 v1beta1，另一个是 v1。
5. 你更新第一个对象。该对象现在以版本 v1 保存，因为 v1 是当前的存储版本。

以前的存储版本 {#previous-storage-versions}

API 服务器在状态字段 storedVersions 中记录曾被标记为存储版本的每个版本。对象可能以任何曾被指定为存储版本的版本保存。存储中不会出现从未成为存储版本的版本的对象。

将现有对象升级到新的存储版本 {#upgrade-existing-objects-to-a-new-stored-version}

弃用版本并删除其支持时，请选择存储升级过程。

选项 1：使用存储版本迁移程序（Storage Version Migrator）

1. 运行存储版本迁移程序
2. 从 CustomResourceDefinition 的 status.storedVersions 字段中去掉老的版本。

选项 2：手动将现有对象升级到新的存储版本

以下是从 v1beta1 升级到 v1 的示例过程。

1. 在 CustomResourceDefinition 文件中将 v1 设置为存储版本，并使用 kubectl 应用它。storedVersions 现在是 v1beta1, v1。
2. 编写升级过程以列出所有现有对象并使用相同内容将其写回存储。这会强制后端使用当前存储版本（即 v1）写入对象。
3. 从 CustomResourceDefinition 的 status.storedVersions 字段中删除 v1beta1。

以下是如何使用 kubectl 为一个 CRD 对象修补 status 子资源的示例：

```
kubectl patch customresourcedefinitions --subresource='status' --type='merge' -p '{"status":{"storedVersions":["v1"]}}'
```